TP Chapitre 3 : Structures et Unions

Programmation C Avancée - L3

Dr. El Hadji Bassirou TOURÉ
Département de Mathématiques et Informatique
Faculté des Sciences et Techniques
Université Cheikh Anta Diop de Dakar

Environnement de travail

Ce TP doit être réalisé dans l'environnement Docker du cours. Si vous n'avez pas encore configuré l'environnement, consultez le TP-1 (Setup Environnement).

Commandes de base:

```
# Linux/macOS
docker run -it -v $(pwd):/workspace elbachirtoure/c-advanced-course:v2.0

# Windows (Git Bash)
docker run -it -v /${PWD}:/workspace elbachirtoure/c-advanced-course:v2.0

# Dans le container, aller dans le workspace
cd /workspace
```

1 Exercice 1 : Manipulation de Structures

1.1 - Création et initialisation

Créer un fichier `ex1_struct.c` avec les structures suivantes:

```c
typedef struct {
    char name[32];
    int age;
    float height;
} Person;

typedef struct {
    int day, month, year;
} Date;

typedef struct {
    Person person;
    Date birth_date;
    char *email;  // allocation dynamique
} Employee;
```

Implémenter:

1. `void print_person(const Person *p)` – affiche les infos
2. `Person create_person(const char *name, int age, float height)`
3. `Employee *create_employee(const char *name, int age, float height, int d, int m, int y, const char *email)`
4. `void free_employee(Employee *e)`

Tester avec:

```c
Employee *e = create_employee("Alice", 30, 1.65, 15, 3, 1994, "alice@test.com");
// afficher, puis libérer
```

1.2 - Comparaison de structures

```c
typedef struct {
    int x, y;
} Point;
```

Implémenter:

1. `int point_equal(const Point *a, const Point *b)` – comparaison membre à membre
2. `int point_equal_memcmp(const Point *a, const Point *b)` – utiliser `memcmp`

Question: Pourquoi `memcmp` peut-il donner des résultats incorrects sur certaines structures ?

2 Exercice 2 : Analyse Mémoire avec GDB

2.1 - Inspection de structures

Créer `ex2_gdb.c`:

```c
#include <stdio.h>
#include <stddef.h>

struct Data {
    char tag;
    int value;
    short code;
    char flags;
};

int main(void) {
    struct Data d = {'A', 0x12345678, 0xABCD, 'F'};

    printf("sizeof(struct Data) = %zu\n", sizeof(struct Data));
    printf("offsetof tag   = %zu\n", offsetof(struct Data, tag));
    printf("offsetof value = %zu\n", offsetof(struct Data, value));
    printf("offsetof code  = %zu\n", offsetof(struct Data, code));
    printf("offsetof flags = %zu\n", offsetof(struct Data, flags));

    return 0;
}
```

Compiler et débugger:

```
gcc -g -o ex2 ex2_gdb.c
gdb ./ex2
```

Commandes GDB à exécuter:

```
(gdb) break main
(gdb) run
(gdb) next
(gdb) ptype struct Data
(gdb) print d
(gdb) print sizeof(d)
(gdb) x/16xb &d
(gdb) print &d.tag
(gdb) print &d.value
```

À rendre:

- Capture du dump hexadécimal (`x/16xb &d`)
- Identification des octets de padding
- Vérification de l'endianness

2.2 - Structures imbriquées

```c
struct Inner {
    char a;
    int b;
};
```

```c

struct Outer {
    char x;
    struct Inner inner;
    char y;
};

int main(void) {
    struct Outer o = {'X', {'A', 42}, 'Y'};
    return 0;
}
```

Avec GDB:

1. Afficher `sizeof(struct Outer)`
2. Examiner le layout avec `x/Nxb &o`
3. Identifier tous les octets de padding
4. Dessiner le schéma mémoire

3 Exercice 3 : Unions et Type Punning

3.1 - Conversion float/int

```c
#include <stdio.h>

union FloatInt {
    float f;
    unsigned int i;
    unsigned char bytes[4];
};

int main(void) {
    union FloatInt u;

    // Tester avec différentes valeurs
    float values[] = {1.0f, -1.0f, 0.0f, 3.14159f, 1e10f};

    for (int j = 0; j < 5; j++) {
        u.f = values[j];
        printf("f = %g\n", u.f);
        printf("  hex: 0x%08X\n", u.i);
        printf("  bytes: ");
        for (int i = 0; i < 4; i++) {
            printf("%02X ", u.bytes[i]);
        }
        printf("\n\n");
    }

    return 0;
}
```

Analyser:

1. L'ordre des octets (little-endian)
2. La structure IEEE 754 (signe, exposant, mantisse)
3. Pourquoi `1.0f` donne `0x3F800000`

3.2 - Tagged Union (Variant)

Implémenter un type variant pouvant contenir `int`, `double`, ou `char*`:

```c
typedef enum { VAR_INT, VAR_DOUBLE, VAR_STRING } VarType;

typedef struct {
    VarType type;
    union {
        int i;
        double d;
        char *s;
    } value;
} Variant;

// À implémenter:
Variant var_int(int i);
Variant var_double(double d);
```

```c
Variant var_string(const char *s);  // alloue une copie
void var_print(const Variant *v);
void var_free(Variant *v);  // libère si string
```

Tester avec un tableau de Variants de types différents.

4 Exercice 4 : Optimisation de Structures

4.1 - Mesure et optimisation

```c
// Version originale
struct Original {
    char a;
    double b;
    char c;
    int d;
    short e;
    char f;
};

// Votre version optimisée
struct Optimized {
    // réorganiser les membres
};
```

Implémenter un programme qui:

1. Affiche `sizeof` et les offsets des deux versions
2. Calcule l'économie en octets et pourcentage
3. Vérifie que les données sont identiques après copie

4.2 - Impact sur les tableaux

```c
#define N 1000000

struct Big { /* version non optimisée */ };
struct Small { /* version optimisée */ };

int main(void) {
    printf("Tableau Big:   %zu Mo\n", N * sizeof(struct Big) / (1024*1024));
    printf("Tableau Small: %zu Mo\n", N * sizeof(struct Small) / (1024*1024));
    return 0;
}
```

Mesurer l'impact mémoire pour 1 million d'éléments.

5 Exercice 5 : Structures Dynamiques

5.1 - Liste chaînée simple

```c
typedef struct Node {
    int data;
    struct Node *next;
} Node;

typedef struct {
    Node *head;
    size_t size;
} List;

// À implémenter:
List *list_create(void);
void list_push_front(List *list, int data);
void list_push_back(List *list, int data);
int list_pop_front(List *list);
void list_print(const List *list);
void list_destroy(List *list);
```

Vérifier avec Valgrind:

```
valgrind --leak-check=full ./ex5
```

5.2 - Flexible Array Member

Implémenter une chaîne de caractères avec sa longueur:

```c
typedef struct {
    size_t length;
    size_t capacity;
    char data[];  // flexible array member
} String;

String *string_create(const char *init);
String *string_concat(const String *a, const String *b);
void string_print(const String *s);
void string_destroy(String *s);
```

6 Projet : Simple Database

Description

Implémenter un système de gestion d'enregistrements en mémoire avec les fonctionnalités suivantes:

- Stockage dynamique de records (tableau redimensionnable)
- Opérations CRUD (Create, Read, Update, Delete)
- Recherche par ID ou par nom
- Affichage formaté

Structure de données

```c
#define MAX_NAME 64
#define MAX_EMAIL 128

typedef struct {
    int id;
    char name[MAX_NAME];
    char email[MAX_EMAIL];
    int age;
    double salary;
} Record;

typedef struct {
    Record *records;
    size_t count;
    size_t capacity;
    int next_id;
} Database;
```

API à implémenter

```c
// Création/Destruction
Database *db_create(size_t initial_capacity);
void db_destroy(Database *db);

// CRUD
int db_insert(Database *db, const char *name, const char *email,
              int age, double salary);  // retourne l'ID
Record *db_find_by_id(Database *db, int id);
Record *db_find_by_name(Database *db, const char *name);
int db_update(Database *db, int id, const char *name,
              const char *email, int age, double salary);
int db_delete(Database *db, int id);

// Affichage
void db_print_all(const Database *db);
void db_print_record(const Record *r);

// Statistiques
size_t db_count(const Database *db);
double db_average_salary(const Database *db);
double db_average_age(const Database *db);
```

Programme de test

```c
int main(void) {
    Database *db = db_create(10);

    // Insertions
    db_insert(db, "Alice", "alice@test.com", 30, 45000.0);
    db_insert(db, "Bob", "bob@test.com", 25, 38000.0);
    db_insert(db, "Charlie", "charlie@test.com", 35, 52000.0);

    printf("=== Tous les enregistrements ===\n");
    db_print_all(db);

    printf("\n=== Recherche par nom: Bob ===\n");
    Record *r = db_find_by_name(db, "Bob");
    if (r) db_print_record(r);

    printf("\n=== Mise a jour ID 2 ===\n");
    db_update(db, 2, "Robert", "robert@test.com", 26, 40000.0);
    db_print_all(db);

    printf("\n=== Statistiques ===\n");
    printf("Nombre: %zu\n", db_count(db));
    printf("Salaire moyen: %.2f\n", db_average_salary(db));
    printf("Age moyen: %.1f\n", db_average_age(db));

    printf("\n=== Suppression ID 1 ===\n");
    db_delete(db, 1);
    db_print_all(db);

    db_destroy(db);
    return 0;
}
```

Contraintes

- Le tableau `records` doit se redimensionner automatiquement (doubler la capacité)
- Aucune fuite mémoire (vérifier avec Valgrind)
- `db_delete` peut réorganiser le tableau (compacter)
- Les ID sont uniques et ne sont jamais réutilisés
- Gestion des erreurs (NULL, ID invalide, etc.)

Soumission

Fichiers à rendre

- `ex1_struct.c` – Exercice 1
- `ex2_gdb.c` + rapport GDB (captures/schémas)
- `ex3_union.c` – Exercice 3
- `ex4_optim.c` – Exercice 4
- `ex5_list.c` – Exercice 5
- `database.h` + `database.c` + `main.c` – Projet
- `Makefile`
- Rapport Valgrind (capture sans erreurs)

Compilation requise:

```
gcc -Wall -Wextra -Werror -std=c11 -g -o prog *.c
```